

宁波大学
NINGBO UNIVERSITY

宁波大学信息学院

实验报告

实验课程名称： 嵌入式系统

学　　院： 信息科学与工程学院

专　　业： 计算机科学与技术

指 导 教 师： 王翀,王艺睿

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 2022 级计算机 2 班

学　　期： 2024-2025 学年第二学期

信息学院实验中心制

实验								
成绩								

实验项目名称	**实验十 Linux 多任务编程实验**

一、实验目的与要求：

1.1 实验目的

(1) 理解 Linux 系统中进程的概念以及进程管理的基本原理。

(2) 学习使用 Linux 系统工具查看和管理系统中的进程。

(3) 掌握 Linux 多任务编程的基本方法，理解进程间的关系和通信方式。

1.2 实验要求

(1) 实践编写守护进程程序，理解守护进程的编写步骤和特性。熟悉使用 ps 命令和/proc 文件系统来查看系统中的进程信息，包括进程 ID、状态、占用资源等。

(2) 编写程序使用 fork、exit 和 exec 系统调用实现多进程程序，理解这些调用在多进程编程中的作用。

(3) 学习进程间通信的基本方式，包括但不限于管道、消息队列、共享内存等，并尝试实现简单的进程间通信。

(4) 理解 Linux 守护进程的概念和特性，学习如何编写和部署一个简单的守护进程。

二、实验设备：

(1) 华清远见开发环境。

(2) 实验平台为华清远见提供的虚拟机环境。

(3) 使用的操作系统是 Ubuntu 12.04 LTS 64-bit。

(4) 虚拟机工具软件为 VMware Player。

三、实验方法（原理、流程图）

实验原理：

1. 使用 ps 命令查看进程信息

ps：查看系统中的进程，Linux 中可以使用 ps -aux 查看所有进程。其中 PID 代表进程 ID，TTY 是该进程是由哪个控制台启动的，CMD 则是命令。

如果想列出更详细的信息，则可使用命令：“ps -auxw”。参数 w 表示加宽显示的命令行，参数 w 可以写多次，通常最多写 3 次，表示加宽 3 次，这足以显示很长的命令行了。

2. 使用 proc 文件系统查看进程信息

/proc 文件系统是一个虚拟文件系统，通过文件系统接口实现对内核的访问，输出系统运行状态。它以文件系统的形式，为操作系统本身和应用进程之间的通信提供了一个界面，使应用程序能够安全，方便的获得系统当前的运行状况和内核的内部数据信息，并且可以修改某些系统的配置信息。

3. 使用 fork、exit 和 exec 系统调用编写多进程程序

fork 后父子进程会同步运行，但父子进程的返回顺序是不确定的。设两个变量 global 和 test 来检测父子进程共享资源的情况。

4. Linux 系统守护进程实验

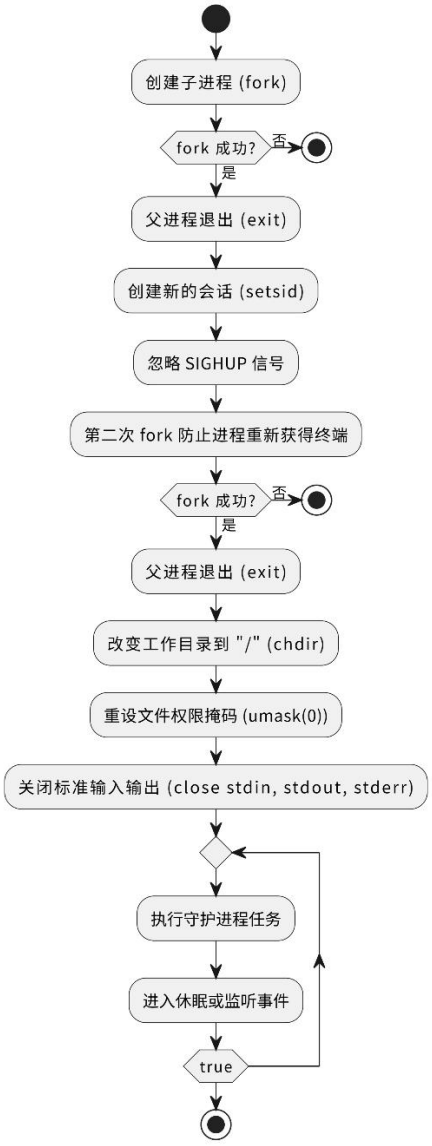
在 Linux 系统中，编写守护进程（Daemon）通常包括以下主要步骤：

1. 创建子进程并退出父进程
    - 使用 fork() 创建子进程，使父进程退出，确保新进程不是终端进程组的领导者。
2. 创建新的会话并成为会话首进程
    - 使用 setsid() 让子进程成为新的会话组长，同时脱离控制终端，避免信号干

扰。

3. 更改工作目录
   - 使用 chdir("/") 将进程的工作目录更改为根目录，防止占用原始挂载点。
4. 重设文件权限掩码
   - 使用 umask(0) 确保守护进程创建的文件不会受到父进程的文件模式屏蔽字影响。
5. 关闭不必要的文件描述符
   - 关闭标准输入（stdin）、标准输出（stdout）和标准错误（stderr），避免守护进程与终端发生交互。
6. 执行守护进程的核心功能
   - 进入主循环，执行需要的任务（如定时操作、监听服务等）。

流程图：



四、实验过程、步骤及内容

4.1 **使用 ps 命令查看进程信息**

ps 是基本的 Linux 命令，通过本实验，不仅要熟悉 ps 命令方法，更重要的是可以了解 Linux 进程的组成。

在 shell 提示符下输入如下命令，并解释输出的结果：

```
$ ps
```

```
root@ubuntu64-vm: /proc
linux@ubuntu64-vm:~$ ps
  PID TTY          TIME CMD
 2584 pts/0    00:00:00 bash
 2696 pts/0    00:00:00 ps
```

终端显示：

```
PID TTY TIME CMD
16767 pts/1 0:00 ps
18029 pts/1 0:00 bash
$ ps aux
PID TTY TIME CMD
```

```
linux@ubuntu64-vm:~$ ps aux
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1  24512  2484 ?        Ss   17:59   0:00 /sbin/init
root         2  0.0  0.0      0     0 ?        S    17:59   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    17:59   0:00 [ksoftirqd/0]
root         6  0.0  0.0      0     0 ?        S    17:59   0:00 [migration/0]
root         7  0.0  0.0      0     0 ?        S    17:59   0:00 [watchdog/0]
root         8  0.0  0.0      0     0 ?        S<   17:59   0:00 [cpuset]
root         9  0.0  0.0      0     0 ?        S<   17:59   0:00 [khelper]
root        10  0.0  0.0      0     0 ?        S    17:59   0:00 [kdevtmpfs]
root        11  0.0  0.0      0     0 ?        S<   17:59   0:00 [netns]
root        12  0.0  0.0      0     0 ?        S    17:59   0:00 [sync_supers]
root        13  0.0  0.0      0     0 ?        S    17:59   0:00 [bdi-default]
root        14  0.0  0.0      0     0 ?        S<   17:59   0:00 [kintegrityd]
root        15  0.0  0.0      0     0 ?        S<   17:59   0:00 [kblockd]
root        16  0.0  0.0      0     0 ?        S<   17:59   0:00 [ata_sff]
root        17  0.0  0.0      0     0 ?        S    17:59   0:00 [khubd]
root        18  0.0  0.0      0     0 ?        S<   17:59   0:00 [md]
root        19  0.0  0.0      0     0 ?        S    17:59   0:00 [kworker/u:1]
root        21  0.0  0.0      0     0 ?        S    17:59   0:00 [khungtaskd]
root        22  0.0  0.0      0     0 ?        S    17:59   0:00 [kswapd0]
root        23  0.0  0.0      0     0 ?        SN   17:59   0:00 [ksmd]
```

4.2 使用 proc 文件系统查看进程信息

本实验将指导学员了解 proc 文件系统，通过 proc 文件系统查询进程信息，可以扩展到修改系统参数。

认识 proc 文件系统的文件和目录：

```
$ cd /proc
$ ls
```

```
linux@ubuntu64-vm:~$ cd /proc
linux@ubuntu64-vm:/proc$ ls
1     1327  2148  2248  2423  3    799        execdomains    partitions
10    1331  2149  2265  2425  321  8          fb             sched_debug
1043  1332  2153  2267  2447  325  801        filesystems    schedstat
1053  1340  2156  2268  2469  35   802        fs             scsi
1062  1370  2157  2271  2473  36   806        interrupts     self
1064  14    2165  2273  2481  37   814        iomem          slabinfo
1067  1424  2169  2284  25    38   817        ioports        softirqs
1085  15    2178  2296  2511  39   834        irq            stat
1087  1503  2187  2298  2522  456  876        kallsyms       swaps
1088  156   2189  23    2524  457  9          kcore          sys
1093  1585  2196  2300  2529  533  910        key-users      sysrq-trigger
11    16    22    2302  2530  553  956        kmsg           sysvipc
1106  1665  2202  2309  2548  59   acpi       kpagecount     timer_list
1189  17    2207  2313  2554  6    asound     kpageflags     timer_stats
1190  1788  2212  2324  2559  60   buddyinfo  latency_stats  tty
12    18    222   2329  2565  641  bus        loadavg        uptime
1255  1863  2221  2347  2574  663  cgroups    locks          version
1275  19    2222  2351  2583  7    cmdline    mdstat         version_signature
```

通过 proc 文件系统查看系统当前进行状态：

```
$ cat /proc/self/status
```

```
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 109 124 1000
VmPeak:     8904 kB
VmSize:     8904 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:       368 kB
VmRSS:       368 kB
VmData:      176 kB
VmStk:       136 kB
VmExe:        44 kB
VmLib:      1928 kB
VmPTE:        40 kB
VmSwap:        0 kB
Threads:        1
SigQ:   0/11915
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: 0000000000000000
SigCgt: 0000000000000000
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: ffffffffffffffff
Cpus_allowed:   ffffffff,ffffffff
Cpus_allowed_list:      0-63
Mems_allowed:   00000000,00000001
Mems_allowed_list:      0
voluntary_ctxt_switches:        0
nonvoluntary_ctxt_switches:     0
```

查询文件句柄的当前使用情况：

```
$ cat /proc/sys/fs/file-nr
426         15252458
```

```
linux@ubuntu64-vm:/proc$ cat /proc/sys/fs/file-nr
6304    0       150474
```

file-nr 文件显示了三个参数：分配的文件句柄总数、当前使用的文件句柄数以及可以分配的最大文件句柄数。如果需要增大/proc/sys/fs/file-max 中的值，请确保正确设置 ulimit 。对于 2.4.20，通常将其设置为 unlimited 。使用 ulimit 命令来验证 ulimit 设置：

```
$ ulimit
unlimited
```

```
linux@ubuntu64-vm:/proc$ ulimit
unlimited
```

通过 proc 文件系统修改内核中预定的一些变量.

修改整个系统中文件句柄的最大数量：

```
$ cat /proc/sys/fs/file-max
52458
$ echo 65536 >/proc/sys/fs/file-max
$ cat /proc/sys/fs/file-max
65536
```

```
root@ubuntu64-vm:/home/linux# cat /proc/sys/fs/file-max
150474
root@ubuntu64-vm:/home/linux# echo 65536 >/proc/sys/fs/file-max
root@ubuntu64-vm:/home/linux# cat /proc/sys/fs/file-max
65536
```

修改网络 TTL：

```
$ cat    /proc/sys/net/ ipv4/ip_default_ttl
64
$ echo 128 >/proc/sys/net/ipv4/ip_default_ttl
$ cat    /proc/sys/net/ ipv4/ip_default_ttl
128
```

```
root@ubuntu64-vm:/proc# cat /proc/sys/net/ipv4/ip_default_ttl
64
root@ubuntu64-vm:/proc# echo 128 >/proc/sys/net/ip_default_ttl
bash: /proc/sys/net/ip_default_ttl: 没有那个文件或目录
root@ubuntu64-vm:/proc# echo 128 >/proc/sys/net/ipv4/ip_default_ttl
root@ubuntu64-vm:/proc# cat /proc/sys/net/ipv4/ip_default_ttl
128
```

修改系统中最大进程数量：

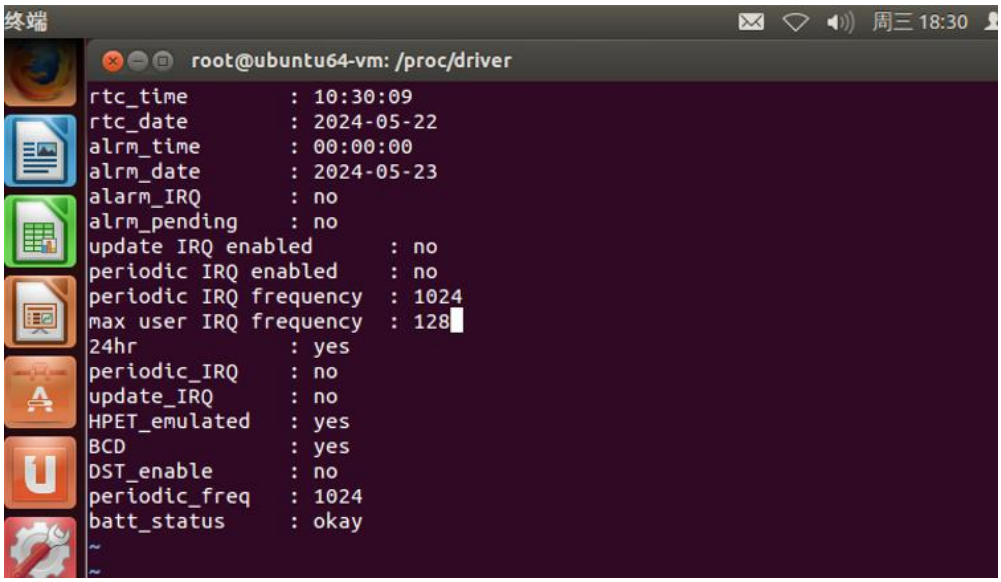
```
$ cat /proc/sys/kernel/pid_max
32768
$ echo 65536 >/proc/sys/kernel/pid_max
$cat /proc/sys/kernel/pid_max
65536
```

```
root@ubuntu64-vm:/proc# cat /proc/sys/kernel/pid_max
32768
root@ubuntu64-vm:/proc# echo 65536 >/proc/sys/kernel/pid_max
root@ubuntu64-vm:/proc# cat /proc/sys/kernel/pid_max
65536
```

修改普通用户的最大 RTC 频率：

```
$ cat /proc/sys/dev/rtc/max-user-freq
64
$ echo 128 >/proc/sys/dev/rtc/max-user-freq
$ cat /proc/sys/dev/rtc/max-user-freq
128
```

```
root@ubuntu64-vm:/proc/sys/dev# cd /proc
root@ubuntu64-vm:/proc# cd driver
root@ubuntu64-vm:/proc/driver# ls
rtc  snd-page-alloc
root@ubuntu64-vm:/proc/driver# cd rtc
bash: cd: rtc: 不是目录
root@ubuntu64-vm:/proc/driver# cat rtc/max-user-freq
cat: rtc/max-user-freq: 不是目录
root@ubuntu64-vm:/proc/driver# cat rtc
rtc_time        : 10:27:33
rtc_date        : 2024-05-22
alrm_time       : 00:00:00
alrm_date       : 2024-05-23
alarm_IRQ       : no
alrm_pending    : no
update IRQ enabled      : no
periodic IRQ enabled    : no
periodic IRQ frequency  : 1024
max user IRQ frequency  : 64
24hr            : yes
periodic_IRQ    : no
update_IRQ      : no
HPET_emulated   : yes
BCD             : yes
DST_enable      : no
periodic_freq   : 1024
batt_status     : okay
```



$ cat /proc/cpuinfo　　//查看 CPU 信息

```
root@ubuntu64-vm: /proc/driver

请按 ENTER 或其它命令继续
root@ubuntu64-vm:/proc/driver# cat /proc/cpuinfo
processor       : 0
vendor_id       : GenuineIntel
cpu family      : 6
model           : 158
model name      : Intel(R) Core(TM) i5-7500 CPU @ 3.40GHz
stepping        : 9
microcode       : 0x5e
cpu MHz         : 3408.179
cache size      : 6144 KB
fpu             : yes
fpu_exception   : yes
cpuid level     : 22
wp              : yes
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov
pat pse36 clflush dts mmx fxsr sse sse2 ss syscall nx pdpe1gb rdtscp lm constant
_tsc up arch_perfmon pebs bts nopl xtopology tsc_reliable nonstop_tsc aperfmperf
 pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt aes xsave a
vx f16c rdrand hypervisor lahf_lm 3dnowprefetch ida arat epb xsaveopt pln pts dt
s fsgsbase smep
bogomips        : 6816.35
```

$ cat /proc/interrupts　　//查看中断信息

```
root@ubuntu64-vm:/proc# cat /proc/interrupts
           CPU0
  0:         58   IO-APIC-edge      timer
  1:       8466   IO-APIC-edge      i8042
  4:       2520   IO-APIC-edge
  6:          3   IO-APIC-edge      floppy
  7:          0   IO-APIC-edge      parport0
  8:          1   IO-APIC-edge      rtc0
  9:          0   IO-APIC-fasteoi   acpi
 12:      42264   IO-APIC-edge      i8042
 14:          0   IO-APIC-edge      ata_piix
 15:       3462   IO-APIC-edge      ata_piix
 16:      36971   IO-APIC-fasteoi   snd_ens1371, vmwgfx
 17:       9089   IO-APIC-fasteoi   ehci_hcd:usb1, ioc0
 18:         79   IO-APIC-fasteoi   uhci_hcd:usb2
 19:      14840   IO-APIC-fasteoi   eth0
 40:          0   PCI-MSI-edge      PCIe PME, pciehp
 41:          0   PCI-MSI-edge      PCIe PME, pciehp
 42:          0   PCI-MSI-edge      PCIe PME, pciehp
 43:          0   PCI-MSI-edge      PCIe PME, pciehp
 44:          0   PCI-MSI-edge      PCIe PME, pciehp
 45:          0   PCI-MSI-edge      PCIe PME, pciehp
 46:          0   PCI-MSI-edge      PCIe PME, pciehp
 47:          0   PCI-MSI-edge      PCIe PME, pciehp
 48:          0   PCI-MSI-edge      PCIe PME, pciehp
 49:          0   PCI-MSI-edge      PCIe PME, pciehp
 50:          0   PCI-MSI-edge      PCIe PME, pciehp
 51:          0   PCI-MSI-edge      PCIe PME, pciehp
 52:          0   PCI-MSI-edge      PCIe PME, pciehp
 53:          0   PCI-MSI-edge      PCIe PME, pciehp
 54:          0   PCI-MSI-edge      PCIe PME, pciehp
 55:          0   PCI-MSI-edge      PCIe PME, pciehp
 56:          0   PCI-MSI-edge      PCIe PME, pciehp
 57:          0   PCI-MSI-edge      PCIe PME, pciehp
 58:          0   PCI-MSI-edge      PCIe PME, pciehp
```

$ cat /proc/ioports　　//设备 IO 端口

```
root@ubuntu64-vm:/proc/driver# cat /proc/ioports
0000-0cf7 : PCI Bus 0000:00
  0000-001f : dma1
  0020-0021 : pic1
  0040-0043 : timer0
  0050-0053 : timer1
  0060-0060 : keyboard
  0064-0064 : keyboard
  0070-0071 : rtc0
  0080-008f : dma page reg
  00a0-00a1 : pic2
  00c0-00df : dma2
  00f0-00ff : fpu
  0170-0177 : 0000:00:07.1
    0170-0177 : ata_piix
  01f0-01f7 : 0000:00:07.1
    01f0-01f7 : ata_piix
  02f8-02ff : serial
  0376-0376 : 0000:00:07.1
```

$ cat /proc/meminfo　　//内存信息

```
root@ubuntu64-vm:/proc/driver# cat /proc/meminfo
MemTotal:        1657464 kB
MemFree:          681908 kB
Buffers:           92796 kB
Cached:           388288 kB
SwapCached:            0 kB
Active:           460324 kB
Inactive:         368976 kB
Active(anon):     348900 kB
Inactive(anon):     2420 kB
Active(file):     111424 kB
Inactive(file):   366556 kB
Unevictable:           0 kB
Mlocked:               0 kB
SwapTotal:       2829308 kB
SwapFree:        2829308 kB
Dirty:                 4 kB
Writeback:             0 kB
AnonPages:        348232 kB
Mapped:           104552 kB
Shmem:              3108 kB
```

$ cat /proc/partitions　　//所有设备的所有分区

```
root@ubuntu64-vm:/proc/driver# cat /proc/partitions
major minor  #blocks  name

  11        0    1048575 sr0
   8        0   83886080 sda
   8        1   81053696 sda1
   8        2          1 sda2
   8        5    2829312 sda5
```

$ cat /proc/pci　　// PCI 设备的信息

```
root@ubuntu64-vm:/proc# cat devices
Character devices:
  1 mem
  4 /dev/vc/0
  4 tty
  4 ttyS
  5 /dev/tty
  5 /dev/console
  5 /dev/ptmx
  5 ttyprintk
  6 lp
  7 vcs
 10 misc
 13 input
 14 sound
 21 sg
 29 fb
 99 ppdev
108 ppp
116 alsa
128 ptm
136 pts
```

$ cat /proc/swaps　　//所有 Swap 分区的信息

```
root@ubuntu64-vm:/proc/driver# cat /proc/swaps
Filename                                Type            Size    Used    Priority
/dev/sda5                               partition       2829308 0       -1
root@ubuntu64-vm:/proc/driver#
```

$ cat /proc/version　　// Linux 的版本号

```
root@ubuntu64-vm:/proc/driver# cat /proc/version
Linux version 3.2.0-24-generic (buildd@yellow) (gcc version 4.6.3 (Ubuntu/Linaro
 4.6.3-1ubuntu5) ) #37-Ubuntu SMP Wed Apr 25 08:43:22 UTC 2012
```

4.3 **使用 fork、exit 和 exec 系统调用编写多进程程序:**

本实验将通过编写 fork 等系统调用的程序，加深对系统进程及其控制的了解。

命令行终端，执行：

$ cd ~/workdir/linux/application/9-process

$ mkdir fork

$ cd fork

```
linux@ubuntu64-vm:~/workdir/linux/application/9-process$ mkdir fork
linux@ubuntu64-vm:~/workdir/linux/application/9-process$ cd fork
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ cp /mnt/hgfs/share
/实验代码1/09.\ Linux系统fork等系统调用实验/实验代码/93.c .
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ ls
93.c
```

将“华清远见-CORTEXA9 资料：\实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\09. Linux 系统 fork 等系统调用实验\实验代码”目录下的“93.c”拷贝到该目录下。

$ gcc 93.c –o fork

$ ./fork　　//执行观察结果

```
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ gcc 93.c -o fork
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ ./fork
the test content!
 fork test!
global=24 test=2 Parent,my PID is 3400
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ global=23 test=1 C
hild,my PID is 3401
```

4.4 **Linux 系统守护进程实验:**

通过实验熟悉守护进程的编写过程。

运行 Ubuntu 系统，打开命令行终端。

执行命令

$ cd ~/workdir/linux/application/9-process

$ mkdir daemon

$ cd daemon

将“华清远见-CORTEXA9 资料：\实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\11. Linux 系统守护进程实验\实验代码”目录下的“init.c”和“test.c”拷贝到该目录下

```
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ cd ..
linux@ubuntu64-vm:~/workdir/linux/application/9-process$ cd daemon/
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ cp /mnt/hgfs/sh
re/实验代码1/11.\ Linux系统守护进程实验/实验代码/ -a .
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ ls
实验代码
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ cp /mnt/hgfs/sh
re/实验代码1/11.\ Linux系统守护进程实验/实验代码/init.c .
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ ls
init.c  实验代码
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ cp /mnt/hgfs/sh
re/实验代码1/11.\ Linux系统守护进程实验/实验代码/test.c .
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ ls
init.c  test.c  实验代码
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$
```

编译执行：

```
$ gcc init.c test.c –o test
$ ./test
```

```
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ gcc init.c test.
c -o test
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ ./test
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 17:59 ?        00:00:00 /sbin/init
root         2     0  0 17:59 ?        00:00:00 [kthreadd]
root         3     2  0 17:59 ?        00:00:00 [ksoftirqd/0]
root         6     2  0 17:59 ?        00:00:00 [migration/0]
root         7     2  0 17:59 ?        00:00:00 [watchdog/0]
root         8     2  0 17:59 ?        00:00:00 [cpuset]
root         9     2  0 17:59 ?        00:00:00 [khelper]
root        10     2  0 17:59 ?        00:00:00 [kdevtmpfs]
root        11     2  0 17:59 ?        00:00:00 [netns]
root        12     2  0 17:59 ?        00:00:00 [sync_supers]
root        13     2  0 17:59 ?        00:00:00 [bdi-default]
root        14     2  0 17:59 ?        00:00:00 [kintegrityd]
root        15     2  0 17:59 ?        00:00:00 [kblockd]
root        16     2  0 17:59 ?        00:00:00 [ata_sff]
```

```
linux@ubuntu64-vm: ~/workdir/linux/application/9-process/daemon
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ cat /tmp/test.lo
g
I'm here at Wed May 22 19:02:45 2024
nI'm here at Wed May 22 19:02:47 2024
nI'm here at Wed May 22 19:02:49 2024
nI'm here at Wed May 22 19:02:51 2024
nI'm here at Wed May 22 19:02:53 2024
nI'm here at Wed May 22 19:02:55 2024
nI'm here at Wed May 22 19:02:57 2024
nI'm here at Wed May 22 19:02:59 2024
nI'm here at Wed May 22 19:03:01 2024
nI'm here at Wed May 22 19:03:03 2024
nI'm here at Wed May 22 19:03:05 2024
nI'm here at Wed May 22 19:03:07 2024
nI'm here at Wed May 22 19:03:09 2024
nI'm here at Wed May 22 19:03:11 2024
nI'm here at Wed May 22 19:03:13 2024
nI'm here at Wed May 22 19:03:15 2024
nI'm here at Wed May 22 19:03:17 2024
nI'm here at Wed May 22 19:03:19 2024
nI'm here at Wed May 22 19:03:21 2024
nI'm here at Wed May 22 19:03:23 2024
nI'm here at Wed May 22 19:03:25 2024
```

实验代码

init.c:

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdlib.h>
void init_daemon(void)
{
    int pid;
    int i;

    if (pid = fork())
    {
        exit(0);
    }  //是父进程，结束父进程

    else if (pid < 0)
    {
        exit( -1 );
    }   //fork 失败，退出

     setsid();  //第一子进程成为新的会话组长和进程组长 并与控制终端分离

    if (pid = fork())
    {
        exit(0);
    } //是第一子进程，结束第一子进程
    else if (pid <  0)
    {
        exit(1);
    }   //fork 失败，退出

    //第二子进程不再是会话组长
    for(i = 0; i < getdtablesize(); ++i)
        //关闭打开的文件描述符
        close(i);
    chdir("/tmp");  //改变工作目录到 /tmp
    umask(0);  //重设文件创建掩模
    return;
}
```

test.c:

```
#include <stdio.h>
#include <time.h>
void init_daemon(void);  //守护进程初始化函数

int main()
{
    FILE *fp;
    time_t  t;
```

```
        init_daemon();  //初始化为 Daemon

        while(1)  //每隔 2 秒钟向 test.log 报告运行状态
        {
            sleep(2);  //睡眠 2 秒钟
            if((fp = fopen("test.log", "a")) != NULL)
            {
                t = time(0);
                fprintf(fp, "I 'm here at %sn", asctime(localtime(&t)) );
                fclose(fp);
            }
        }
        return 0;
}
```

五、实验数据（现象）处理分析

5.1 使用 fork、exit 和 exec 系统调用编写多进程程序：

```
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ gcc 93.c -o fork
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ ./fork
the test content!
 fork test!
global=24 test=2 Parent,my PID is 3400
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ global=23 test=1 C
hild,my PID is 3401
```

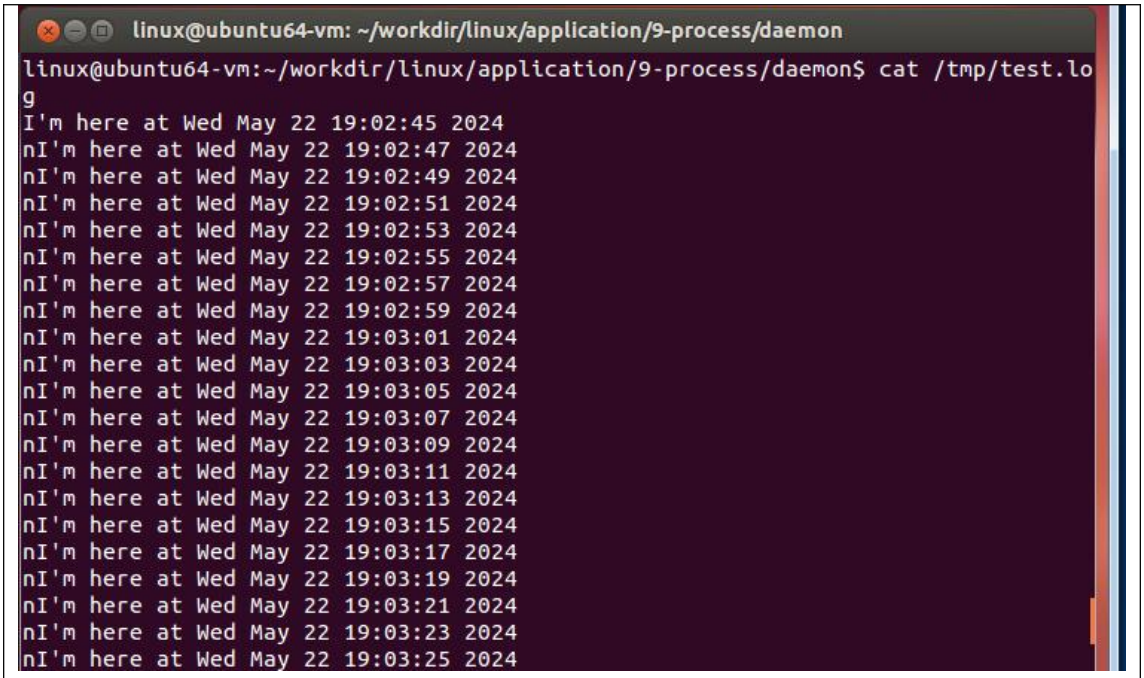
可以看出父子进程打印出了各自的进程号和对应变量的值，显然 global 和 test 在父子进程间是独立的，其各自的操作不会对对方的值有影响。

5.2 Linux 系统守护进程实验

```
    $ ps   -ef          //查看进程
```

```
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ gcc init.c test.
c -o test
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ ./test
linux@ubuntu64-vm:~/workdir/linux/application/9-process/daemon$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 17:59 ?        00:00:00 /sbin/init
root         2     0  0 17:59 ?        00:00:00 [kthreadd]
root         3     2  0 17:59 ?        00:00:00 [ksoftirqd/0]
root         6     2  0 17:59 ?        00:00:00 [migration/0]
root         7     2  0 17:59 ?        00:00:00 [watchdog/0]
root         8     2  0 17:59 ?        00:00:00 [cpuset]
root         9     2  0 17:59 ?        00:00:00 [khelper]
root        10     2  0 17:59 ?        00:00:00 [kdevtmpfs]
root        11     2  0 17:59 ?        00:00:00 [netns]
root        12     2  0 17:59 ?        00:00:00 [sync_supers]
root        13     2  0 17:59 ?        00:00:00 [bdi-default]
root        14     2  0 17:59 ?        00:00:00 [kintegrityd]
root        15     2  0 17:59 ?        00:00:00 [kblockd]
root        16     2  0 17:59 ?        00:00:00 [ata_sff]
```

打开/tmp/test.log 可以看到程序的输出内容，从输出可以发现 test 守护进程的各种特性满足上面的要求。



六、实验结论：

在本次实验中，我深入学习了 Linux 多任务编程的核心概念，并通过一系列实验操作加深了对进程管理的理解。首先，通过使用 ps 命令，我掌握了如何查看系统中当前运行的进程信息，包括进程 ID（PID）、终端（TTY）、执行时间（TIME）和命令（CMD）等关键字段。结合 ps -aux 命令，我能够获取系统中所有进程的详细列表，从而提高了对进程状态的监控能力。这一实验使我对 Linux 进程管理的基本原理有了更加清晰的认识。

其次，通过探索 /proc 文件系统，我了解了 Linux 内核提供的虚拟文件系统的作用。实验过程中，我使用 cat /proc/self/status 等命令查询了进程的详细信息，并通过修改 /proc/sys/fs/file-max 等系统参数，成功调整了系统资源的分配。这使我意识到 /proc 文件系统不仅是一个重要的系统监控工具，同时也是调整和优化系统性能的便捷方式。

在多进程编程实验中，我使用 fork() 创建了子进程，并通过 exit() 和 exec() 控制进程的生命周期。实验结果表明，父子进程虽然共享代码段，但它们的变量和数据是相互独立的，修改一个进程的变量不会影响另一个进程。这一实验让我深刻理解了进程的隔离性以及进程间同步的复杂性，也为后续学习进程间通信（IPC）和线程管理奠定了基础。

最后，在守护进程实验中，我成功编写了一个标准的 Linux 守护进程，并验证了其运行特性。通过 setsid() 使进程独立于终端、调整工作目录、修改文件权限掩码，以及关闭标准输入输出等操作，我掌握了守护进程的创建方法。运行守护进程后，我通过日志文件验证了其持续运行的特性，并学会了如何在后台执行系统任务。这一实验让我理解了 Linux 守护进程在服务管理中的重要性，为未来编写长期运行的系统服务提供了实践经验。

综上所述，本次实验不仅加深了我对 Linux 多任务编程的理解，还让我掌握了多个关键的进程管理工具和系统调用，提升了在嵌入式系统开发中的实践能力。

七、指导教师批阅意见：

成绩评定： 指导教师签字： 年　　月　　日
八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。